

***“PageTurners Bookstore –  
Data Management using SQL Server”***





**FACULTY OF MANAGEMENT  
SIR PADAMPAT SINGHANIA UNIVERSITY  
UDAIPUR – 313601, RAJASTHAN, INDIA**

**REPORT**

**On**

**“PageTurners Bookstore-- Data Management using SQL Server”**

**MASTER OF BUSINESS ADMINISTRATION**

***By***

**Student Name: Pooja Kumari Luhar**

**Enrolment No. 24MBA00500**

***Under the supervision of***

**Prof. Gour Saha**

## 1. History of PageTurners Bookstore

The store was established in 2010 in a mid-sized city of Bhilwara, Rajasthan with the aim of providing a wide range of books to students, professionals, and casual readers. Over time, it has grown from a small local shop into a well-organized PageTurners Bookstore offering genres such as Fiction, Science, Technology, and Business.

With increasing sales and customers, manual record-keeping became difficult. To solve this, the owner decided to use a database system. Implementing the PageTurners Bookstore's data in SQL Server ensures proper management of books, authors, customers, and orders, while also helping in sales analysis and business planning.

## 2. Background of the Study

PageTurners Bookstore handles a lot of data about books, authors, customers, orders, and suppliers. PageTurners Bookstores keep these records manually or in Excel sheets. This creates several problems such as:

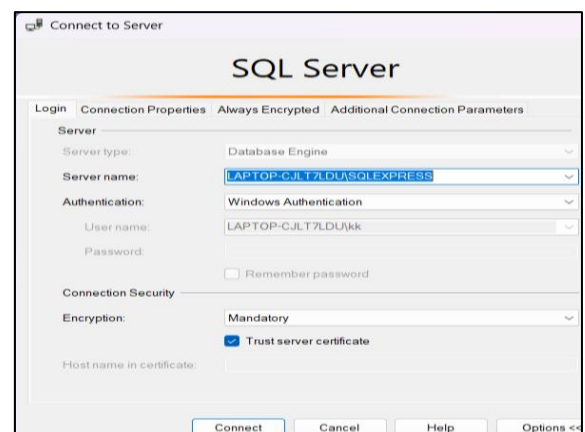
- Difficulty in tracking inventory (which books are available or out of stock).
- No clear way to find top-selling books or best authors.
- Problems in analyzing customer purchase patterns (e.g., which customers buy the most or which countries prefer certain authors).
- Lack of clarity about revenue by genre or profitability of books.
- Slow and confusing report generation for decision-making.



Because of these issues, the PageTurners Bookstore cannot take timely, data-driven decisions about restocking, pricing, or marketing.

To solve these problems, this project uses **SQL Server Management Studio (SSMS)** to design a structured database system. Our purpose to solve queries through SQL is to:

- Store and manage data properly in tables.
- Run queries to extract useful information.
- Identify sales patterns like top-selling books or most profitable genres.
- Update records efficiently (e.g., increase prices by 10%).
- Generate clear reports for better business decisions.



## 2. Objectives of the Project

The aim of this project is to design a database for **PageTurners Bookstore** and use SQL Server to solve the owner's business problems.



To begin with, we conducted an **interview with the bookstore owner** to understand the challenges faced in managing sales, customers, books, and suppliers. From this discussion, we identified several important business questions that the owner wants to address. Additionally, during our analysis, we identified more queries that could provide valuable insights.

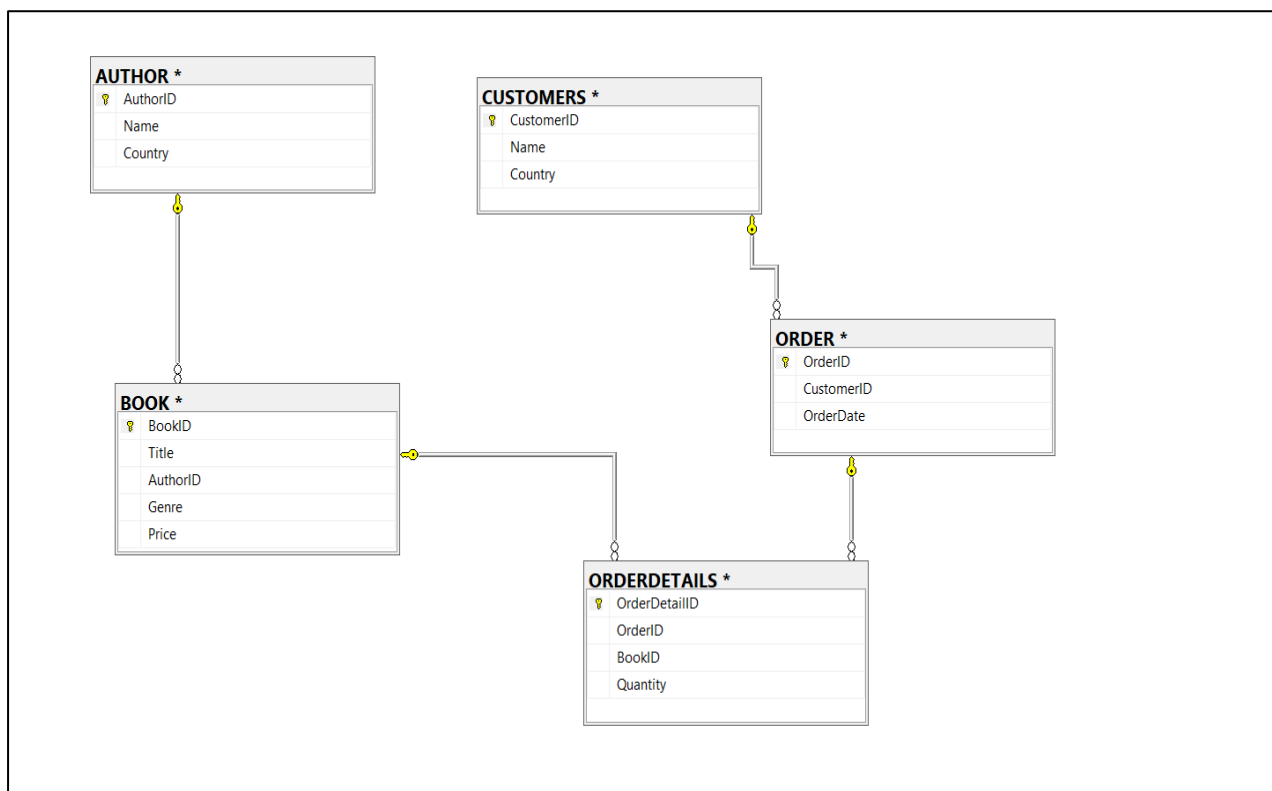
Through this project, we aim to:

- Create tables for authors, books, customers, suppliers, and orders.
- Clean the data by removing duplicate values and checking for missing (NULL) values.
- Solve the bookstore owner's key queries using SQL, such as:
  - What are all the books and their prices?
  - Which customers bought books by authors from the UK?
  - How many books are there in each genre (Fantasy, Fiction, Biography, Literary)?
  - Which books cost more than \$14?
  - Which customers purchased which books?
  - What are all the books along with their authors and genres?
  - Which customers ordered books costing more than \$12?
  - What are the top 3 best-selling books by quantity?
  - What is the total revenue earned per genre?
  - What is the average price of books per author?

By addressing these queries with SQL queries, the project will help the PageTurners Bookstore owner make better decisions related to **stock management, pricing strategies, marketing campaigns, and customer relationship management**.

### 3. Database Design

To better understand how different tables are connected in the database, we created a **Relational Diagram**. This diagram visually represents the structure of the database and shows how the entities (tables) such as authors, books, customers, orders and OrderDetails are linked with each other through relationships (primary keys and foreign keys).



The following points show the relationships between the tables in the **Relational Diagram**:

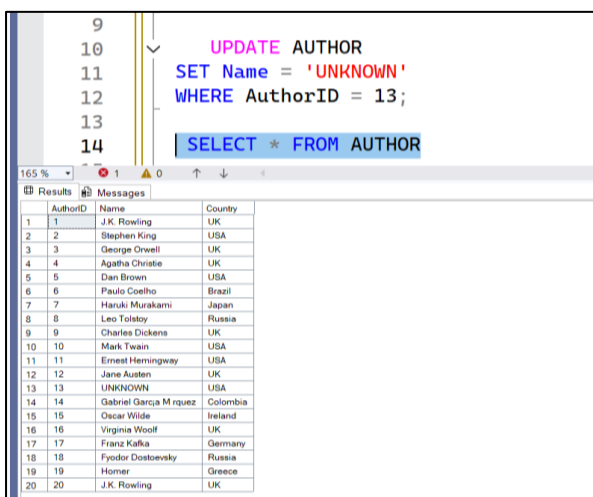
- ⊙ **AUTHOR** → **BOOK**: Each author can write multiple books. This is a one-to-many relationship, where one AuthorID maps to many Book records.
- ⊙ **CUSTOMERS** → **ORDER**: Each customer can place multiple orders. This is a one-to-many relationship, where one CustomerID is linked to many orders.
- ⊙ **ORDER** → **ORDERDETAILS**: Each order can contain multiple books. The OrderDetails table acts as a bridge, storing which books and how many were purchased in each order.
- ⊙ **BOOK** → **ORDERDETAILS**: Each book can appear in multiple orders. This shows a many-to-many relationship between Books and Orders, resolved through the OrderDetails table.

## 4. Methodology

Before describing the step-by-step work, the following lists the kinds of SQL queries and features we used in the PageTurners Bookstore project, explanation of how each was applied.

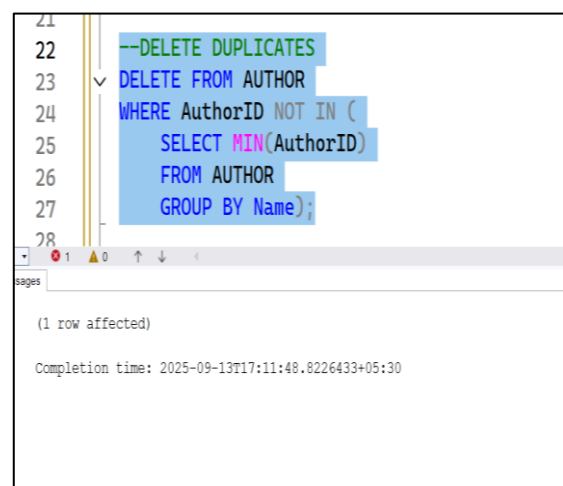
### DML (Data Manipulation Language) — INSERT, UPDATE, DELETE

Used to populate tables with initial/sample data, correct data errors, update missing fields, and remove duplicate or invalid rows during the cleaning phase.



```
9
10 UPDATE AUTHOR
11 SET Name = 'UNKNOWN'
12 WHERE AuthorID = 13;
13
14 SELECT * FROM AUTHOR
```

| AuthorID | Name                   | Country  |
|----------|------------------------|----------|
| 1        | J.K. Rowling           | UK       |
| 2        | Stephen King           | USA      |
| 3        | George Orwell          | UK       |
| 4        | Agatha Christie        | UK       |
| 5        | Dan Brown              | USA      |
| 6        | Paulo Coelho           | Brazil   |
| 7        | Haruki Murakami        | Japan    |
| 8        | Leo Tolstoy            | Russia   |
| 9        | Charles Dickens        | UK       |
| 10       | Mark Twain             | USA      |
| 11       | Ernest Hemingway       | USA      |
| 12       | Jane Austen            | UK       |
| 13       | UNKNOWN                | USA      |
| 14       | Gabriel Garcia Marquez | Colombia |
| 15       | Oscar Wilde            | Ireland  |
| 16       | Virginia Woolf         | UK       |
| 17       | Franz Kafka            | Germany  |
| 18       | Fyodor Dostoevsky      | Russia   |
| 19       | Homer                  | Greece   |
| 20       | J.K. Rowling           | UK       |



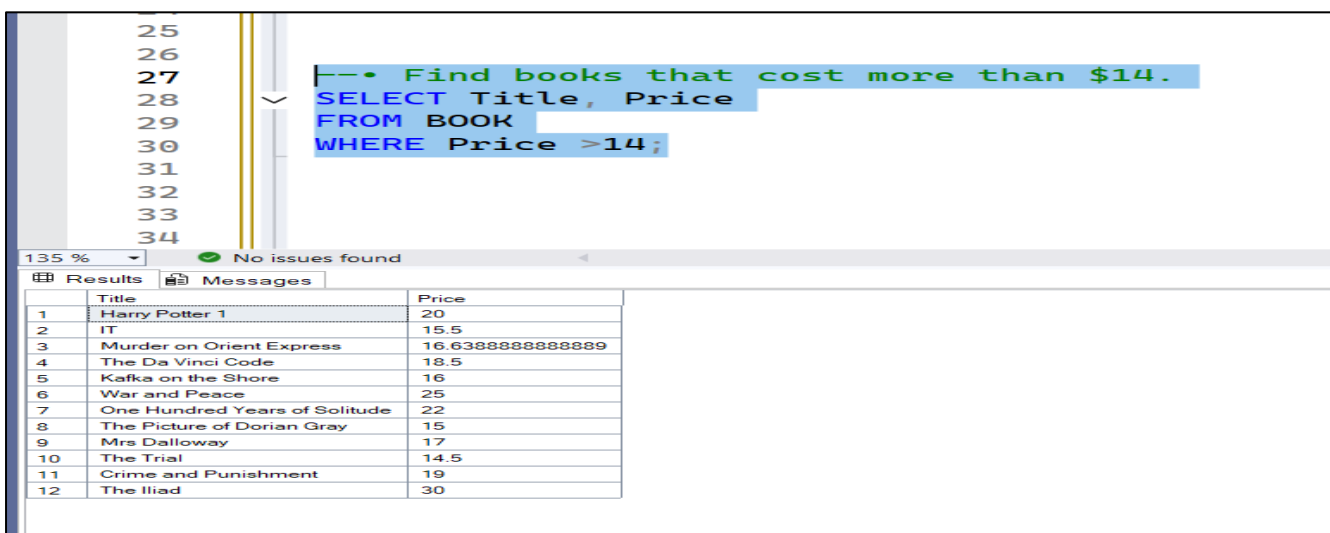
```
21
22 --DELETE DUPLICATES
23 DELETE FROM AUTHOR
24 WHERE AuthorID NOT IN (
25 SELECT MIN(AuthorID)
26 FROM AUTHOR
27 GROUP BY Name);
28
```

(1 row affected)

Completion time: 2025-09-13T17:11:48.8226433+05:30

### DQL / SELECT queries — SELECT ... FROM ... WHERE ... (with JOIN, GROUP BY, HAVING, ORDER BY)

The core of the analysis: retrieve lists of books and prices, find customers who bought books by UK authors, compute counts per genre, filter books by price, and list which customers bought which books.

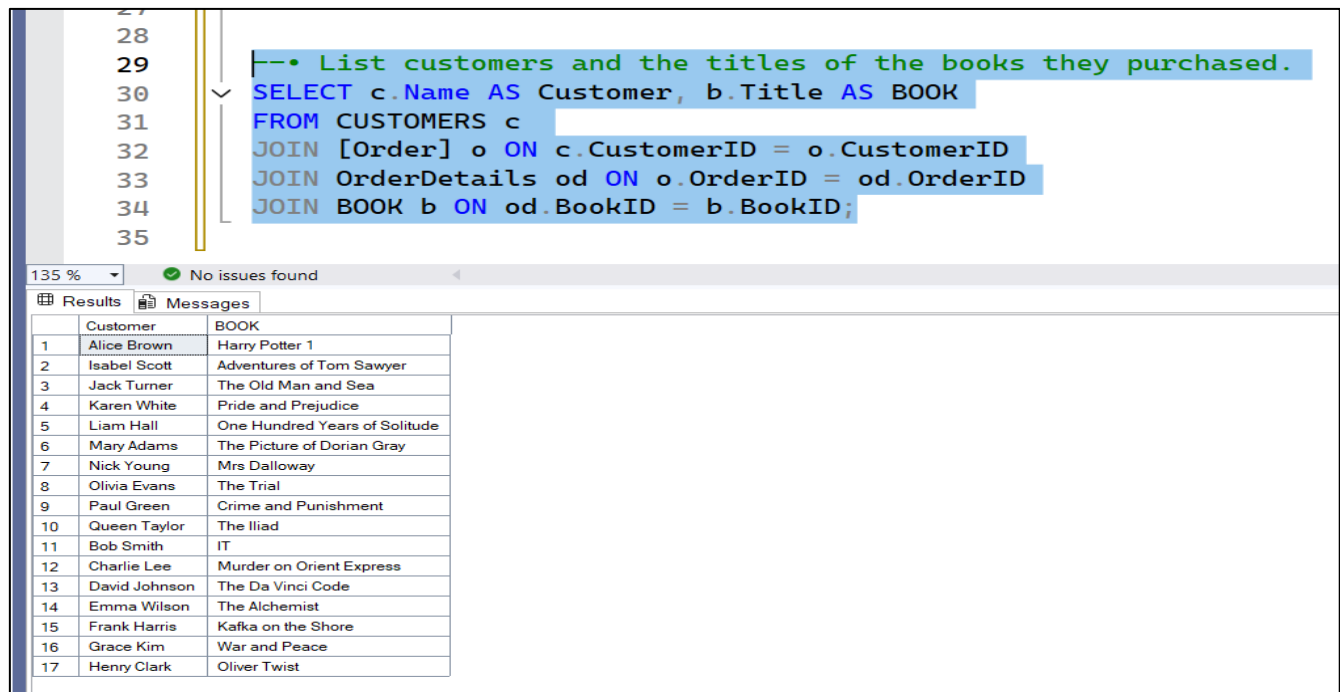


```
25
26
27 --• Find books that cost more than $14.
28 SELECT Title, Price
29 FROM BOOK
30 WHERE Price >14;
31
32
33
34
```

| Title                           | Price             |
|---------------------------------|-------------------|
| 1 Harry Potter 1                | 20                |
| 2 IT                            | 15.5              |
| 3 Murder on Orient Express      | 16.63888888888889 |
| 4 The Da Vinci Code             | 18.5              |
| 5 Kafka on the Shore            | 16                |
| 6 War and Peace                 | 25                |
| 7 One Hundred Years of Solitude | 22                |
| 8 The Picture of Dorian Gray    | 15                |
| 9 Mrs Dalloway                  | 17                |
| 10 The Trial                    | 14.5              |
| 11 Crime and Punishment         | 19                |
| 12 The Iliad                    | 30                |

## Joins & Relationship Queries — INNER JOIN, LEFT JOIN, RIGHT JOIN

Used to combine data across tables (e.g., Books ↔ Authors, Orders ↔ OrderDetails ↔ Books, Customers ↔ Orders) so we can answer cross-table business questions.



The screenshot shows a SQL query in the SQL Server Enterprise Manager interface. The query is designed to list customers and the titles of the books they purchased. The query text is as follows:

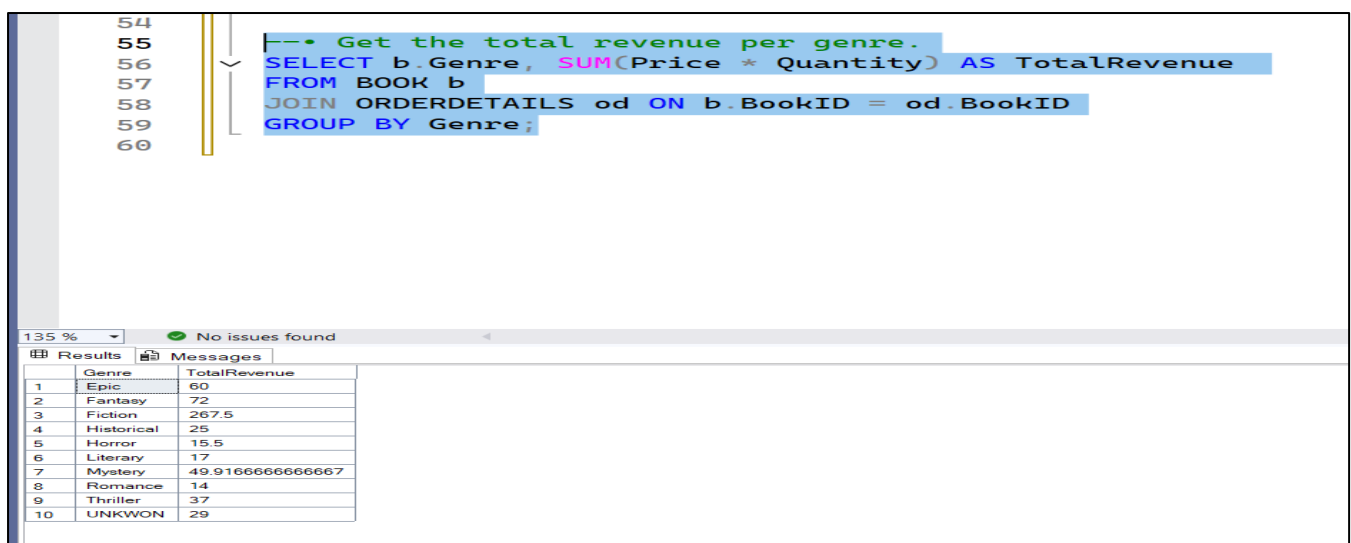
```
--• List customers and the titles of the books they purchased.
SELECT c.Name AS Customer, b.Title AS BOOK
FROM CUSTOMERS c
JOIN [Order] o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN BOOK b ON od.BookID = b.BookID;
```

The results pane shows a table with two columns: Customer and BOOK. The data is as follows:

|    | Customer      | BOOK                          |
|----|---------------|-------------------------------|
| 1  | Alice Brown   | Harry Potter 1                |
| 2  | Isabel Scott  | Adventures of Tom Sawyer      |
| 3  | Jack Turner   | The Old Man and Sea           |
| 4  | Karen White   | Pride and Prejudice           |
| 5  | Liam Hall     | One Hundred Years of Solitude |
| 6  | Mary Adams    | The Picture of Dorian Gray    |
| 7  | Nick Young    | Mrs Dalloway                  |
| 8  | Olivia Evans  | The Trial                     |
| 9  | Paul Green    | Crime and Punishment          |
| 10 | Queen Taylor  | The Iliad                     |
| 11 | Bob Smith     | IT                            |
| 12 | Charlie Lee   | Murder on Orient Express      |
| 13 | David Johnson | The Da Vinci Code             |
| 14 | Emma Wilson   | The Alchemist                 |
| 15 | Frank Harris  | Kafka on the Shore            |
| 16 | Grace Kim     | War and Peace                 |
| 17 | Henry Clark   | Oliver Twist                  |

## Aggregate functions & grouping — COUNT(), SUM(), AVG(), MAX(), MIN()

Used to calculate genre-wise book counts, total revenue per genre, average price per author, and top-selling titles.



The screenshot shows a SQL query in the SQL Server Enterprise Manager interface. The query is designed to get the total revenue per genre. The query text is as follows:

```
--• Get the total revenue per genre.
SELECT b.Genre, SUM(Price * Quantity) AS TotalRevenue
FROM BOOK b
JOIN ORDERDETAILS od ON b.BookID = od.BookID
GROUP BY Genre;
```

The results pane shows a table with two columns: Genre and TotalRevenue. The data is as follows:

|    | Genre      | TotalRevenue     |
|----|------------|------------------|
| 1  | Epic       | 60               |
| 2  | Fantasy    | 72               |
| 3  | Fiction    | 267.5            |
| 4  | Historical | 25               |
| 5  | Horror     | 15.5             |
| 6  | Literary   | 17               |
| 7  | Mystery    | 49.9166666666667 |
| 8  | Romance    | 14               |
| 9  | Thriller   | 37               |
| 10 | UNKWON     | 29               |

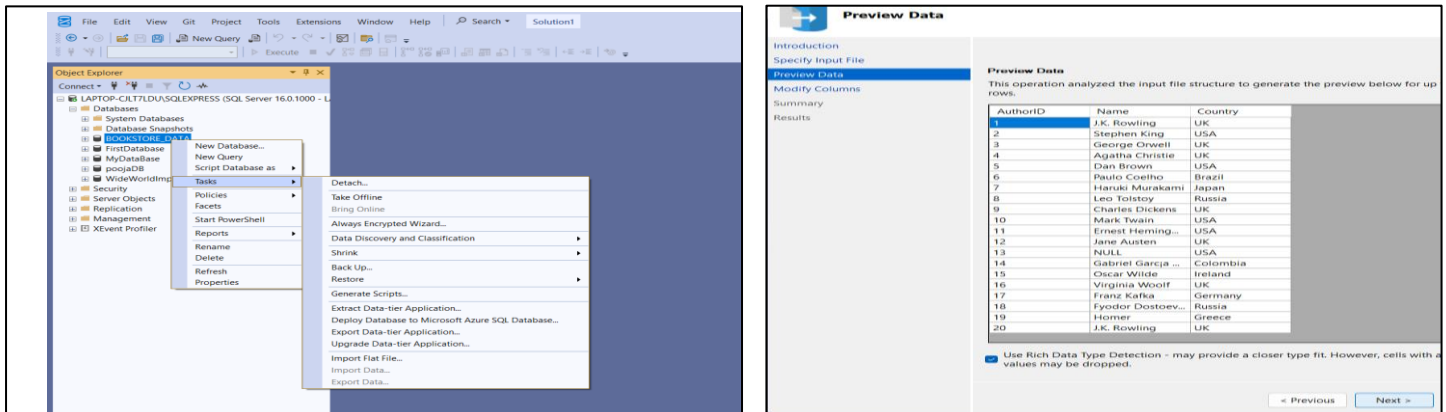
## 4.1 Database Creation

- A new database named **Bookstore** was created.
- Tables were planned for *Authors*, *Books*, *Customers*, *Orders*, *OrderDetails*.
- Each table had proper columns such as BookID, AuthorID, CustomerID, Price, and Genre.

## 4.2 Data Insertion (Importing Excel Data into SQL Server)

- The sample data was first prepared in **Excel sheets** (for example, Authors.xlsx, Books.xlsx, Customers. etc.).
- In SQL Server Management Studio, we clicked on the **Bookstore database**, then selected **Tasks** → **Import Data**.
- A new screen appeared where we chose **Excel** as the data source.

- We clicked **Browse** and selected the Excel file (for example, Authors).
- we mapped the Excel columns with SQL Server table columns and set options like **Primary Key** and whether a column allows **NULL or NOT NULL** values.
- SQL Server processed the file, and finally the data was successfully inserted into the database tables.



### 4.3 Data Cleaning

While working on the PageTurners Bookstore database, two main issues were identified: **missing values (NULLs)** and **duplicate records**. These problems can create confusion and reduce the accuracy of reports if not handled properly.

In some cases, missing values may not have a significant impact. For example, if a customer's middle name is missing, it does not affect sales analysis or revenue reports. However, in other cases, missing values can directly influence results. For instance, if the price of a book or the quantity sold is missing, calculations of revenue or profit will be incorrect. Similarly, duplicate records may lead to inflated counts of customers, books, or sales, giving misleading insights.

Therefore, to ensure data quality and reliability, SQL queries were applied to clean the database. This included replacing or handling NULL values where they impact analysis, and removing duplicate records to maintain consistency. By doing this, the database becomes more accurate and trustworthy, ensuring that reports reflect the true performance of the bookstore.

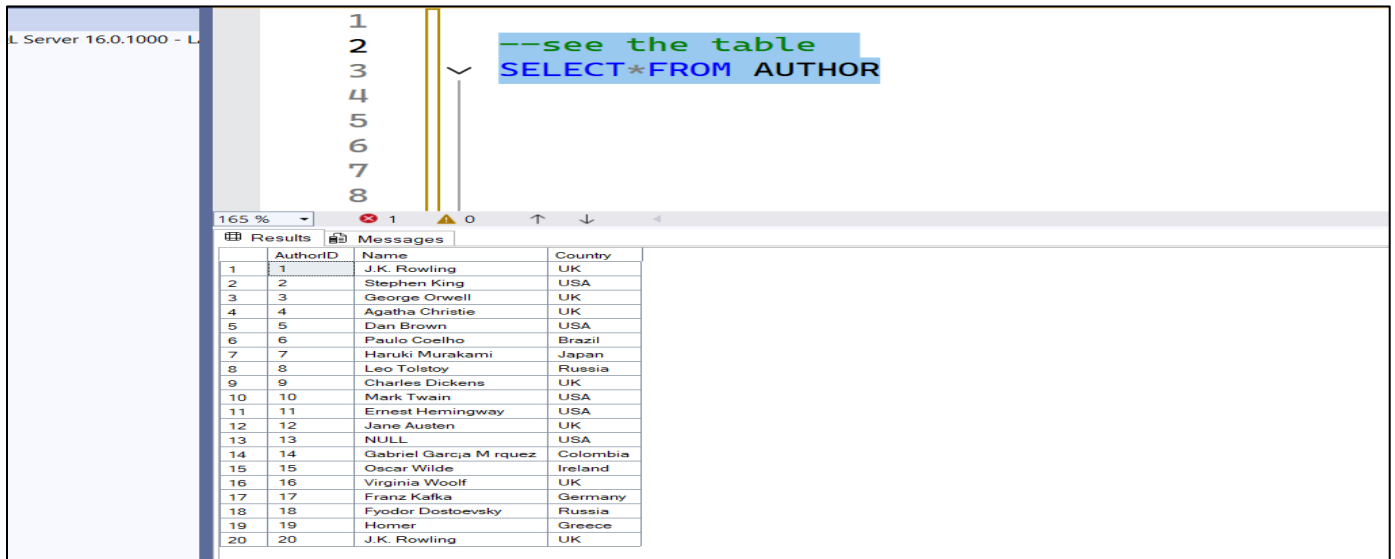
#### 4.3.1. Handling Null Values

- Some records had missing details, such as customers without a country name or books without prices.
- To check for null values, we used the following query:

#### **Authors Table —**

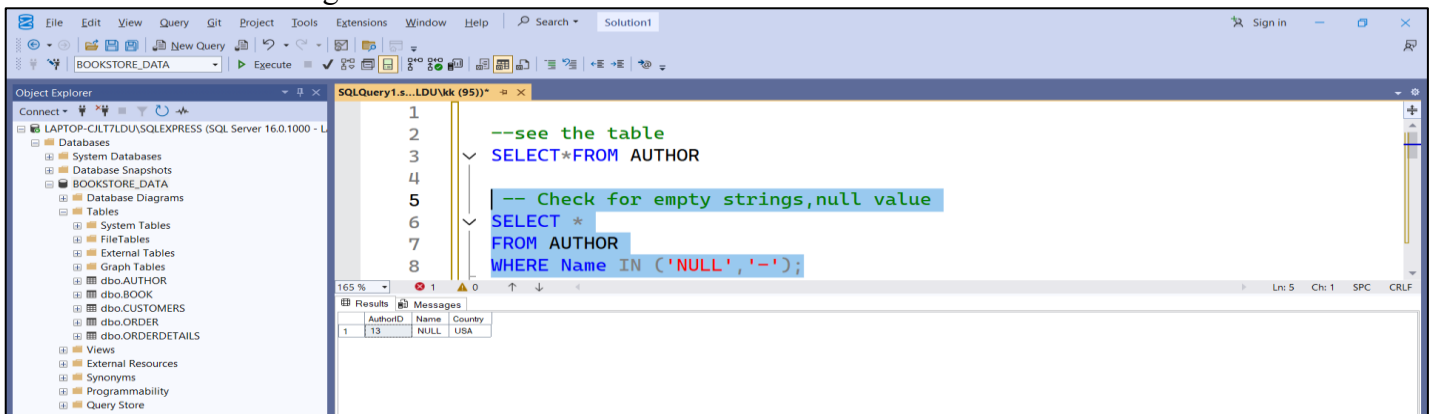
- **Problem Found:** Some rows had missing or placeholder values (NULL, -) in the *Name* column. Duplicate author names also appeared.
- **Action Taken:**
  - ✓ Updated missing/invalid author names to “UNKNOWN”.
  - ✓ Removed duplicate author entries, keeping only one record for each name.

--Before Clean the Author Table view



| AuthorID | Name                   | Country  |
|----------|------------------------|----------|
| 1        | J.K. Rowling           | UK       |
| 2        | Stephen King           | USA      |
| 3        | George Orwell          | UK       |
| 4        | Agatha Christie        | UK       |
| 5        | Dan Brown              | USA      |
| 6        | Paulo Coelho           | Brazil   |
| 7        | Haruki Murakami        | Japan    |
| 8        | Leo Tolstoy            | Russia   |
| 9        | Charles Dickens        | UK       |
| 10       | Mark Twain             | USA      |
| 11       | Ernest Hemingway       | USA      |
| 12       | Jane Austen            | UK       |
| 13       | NULL                   | USA      |
| 14       | Gabriel Garcia M rquez | Colombia |
| 15       | Oscar Wilde            | Ireland  |
| 16       | Virginia Woolf         | UK       |
| 17       | Franz Kafka            | Germany  |
| 18       | Fyodor Dostoevsky      | Russia   |
| 19       | Homer                  | Greece   |
| 20       | J.K. Rowling           | UK       |

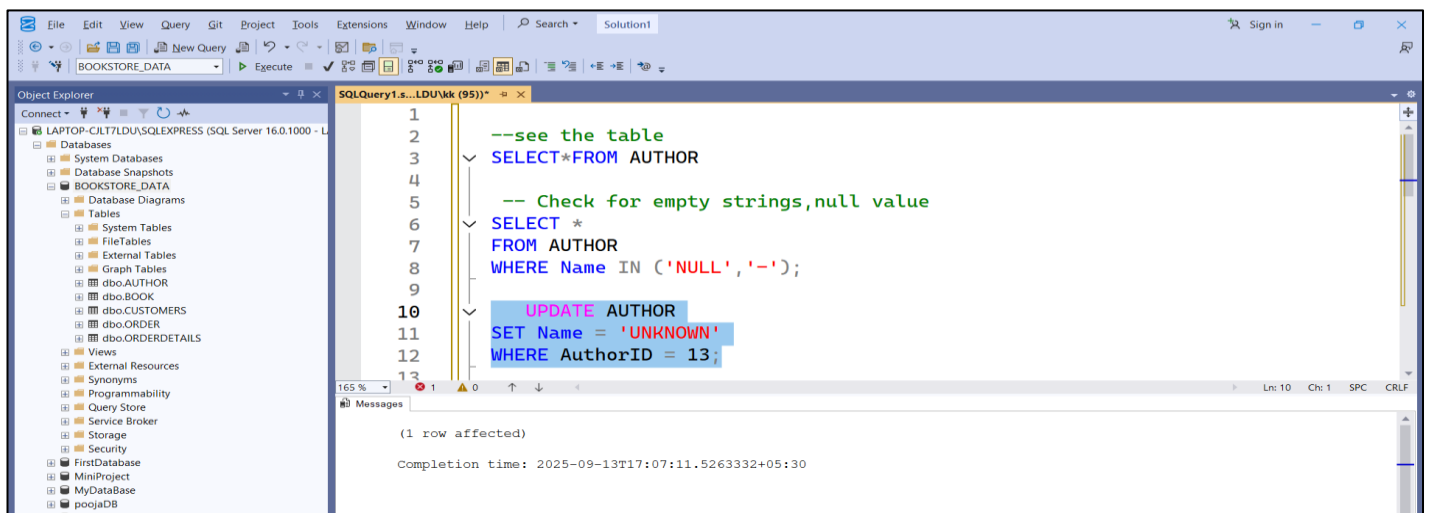
-- Find rows with missing values in Author table



| AuthorID | Name                   | Country  |
|----------|------------------------|----------|
| 1        | J.K. Rowling           | UK       |
| 2        | Stephen King           | USA      |
| 3        | George Orwell          | UK       |
| 4        | Agatha Christie        | UK       |
| 5        | Dan Brown              | USA      |
| 6        | Paulo Coelho           | Brazil   |
| 7        | Haruki Murakami        | Japan    |
| 8        | Leo Tolstoy            | Russia   |
| 9        | Charles Dickens        | UK       |
| 10       | Mark Twain             | USA      |
| 11       | Ernest Hemingway       | USA      |
| 12       | Jane Austen            | UK       |
| 13       | NULL                   | USA      |
| 14       | Gabriel Garcia M rquez | Colombia |
| 15       | Oscar Wilde            | Ireland  |
| 16       | Virginia Woolf         | UK       |
| 17       | Franz Kafka            | Germany  |
| 18       | Fyodor Dostoevsky      | Russia   |
| 19       | Homer                  | Greece   |
| 20       | J.K. Rowling           | UK       |

- Once identified, we either **updated the missing data** with default values or,
- if the row was not useful, **deleted it**.

-- Updating null values:



| AuthorID | Name                   | Country  |
|----------|------------------------|----------|
| 1        | J.K. Rowling           | UK       |
| 2        | Stephen King           | USA      |
| 3        | George Orwell          | UK       |
| 4        | Agatha Christie        | UK       |
| 5        | Dan Brown              | USA      |
| 6        | Paulo Coelho           | Brazil   |
| 7        | Haruki Murakami        | Japan    |
| 8        | Leo Tolstoy            | Russia   |
| 9        | Charles Dickens        | UK       |
| 10       | Mark Twain             | USA      |
| 11       | Ernest Hemingway       | USA      |
| 12       | Jane Austen            | UK       |
| 13       | UNKNOWN                | USA      |
| 14       | Gabriel Garcia M rquez | Colombia |
| 15       | Oscar Wilde            | Ireland  |
| 16       | Virginia Woolf         | UK       |
| 17       | Franz Kafka            | Germany  |
| 18       | Fyodor Dostoevsky      | Russia   |
| 19       | Homer                  | Greece   |
| 20       | J.K. Rowling           | UK       |



## --After View Author Table

```
SELECT * FROM AUTHOR
WHERE Name IN ('NULL', '-');

UPDATE AUTHOR
SET Name = 'UNKNOWN'
WHERE AuthorID = 13;

SELECT * FROM AUTHOR
```

| AuthorID | Name                   | Country  |
|----------|------------------------|----------|
| 1        | J.K. Rowling           | UK       |
| 2        | Stephen King           | USA      |
| 3        | George Orwell          | UK       |
| 4        | Agatha Christie        | UK       |
| 5        | Dan Brown              | USA      |
| 6        | Paulo Coelho           | Brazil   |
| 7        | Haruki Murakami        | Japan    |
| 8        | Leo Tolstoy            | Russia   |
| 9        | Charles Dickens        | UK       |
| 10       | Mark Twain             | USA      |
| 11       | Ernest Hemingway       | USA      |
| 12       | Jane Austen            | UK       |
| 13       | UNKNOWN                | USA      |
| 14       | Gabriel Garcia Marquez | Colombia |
| 15       | Oscar Wilde            | Ireland  |
| 16       | Virginia Woolf         | UK       |
| 17       | Franz Kafka            | Germany  |
| 18       | Fyodor Dostoevsky      | Russia   |
| 19       | Homer                  | Greece   |
| 20       | J.K. Rowling           | UK       |

### 4.3.2. Removing Duplicate Records

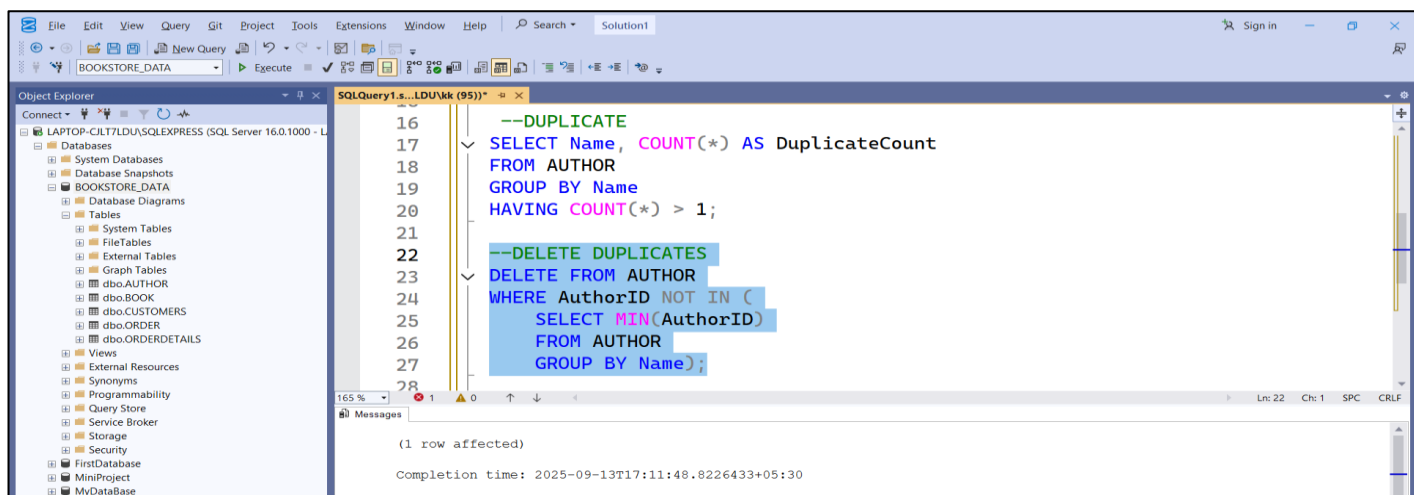
- Some tables contained the same record more than once (e.g., the same customer repeated).
- To find duplicates, we used:
- -- Find duplicate

```
SELECT * FROM AUTHOR

--DUPLICATE
SELECT Name, COUNT(*) AS DuplicateCount
FROM AUTHOR
GROUP BY Name
HAVING COUNT(*) > 1;
```

| Name         | DuplicateCount |
|--------------|----------------|
| J.K. Rowling | 2              |

-- Remove duplicate customer records-

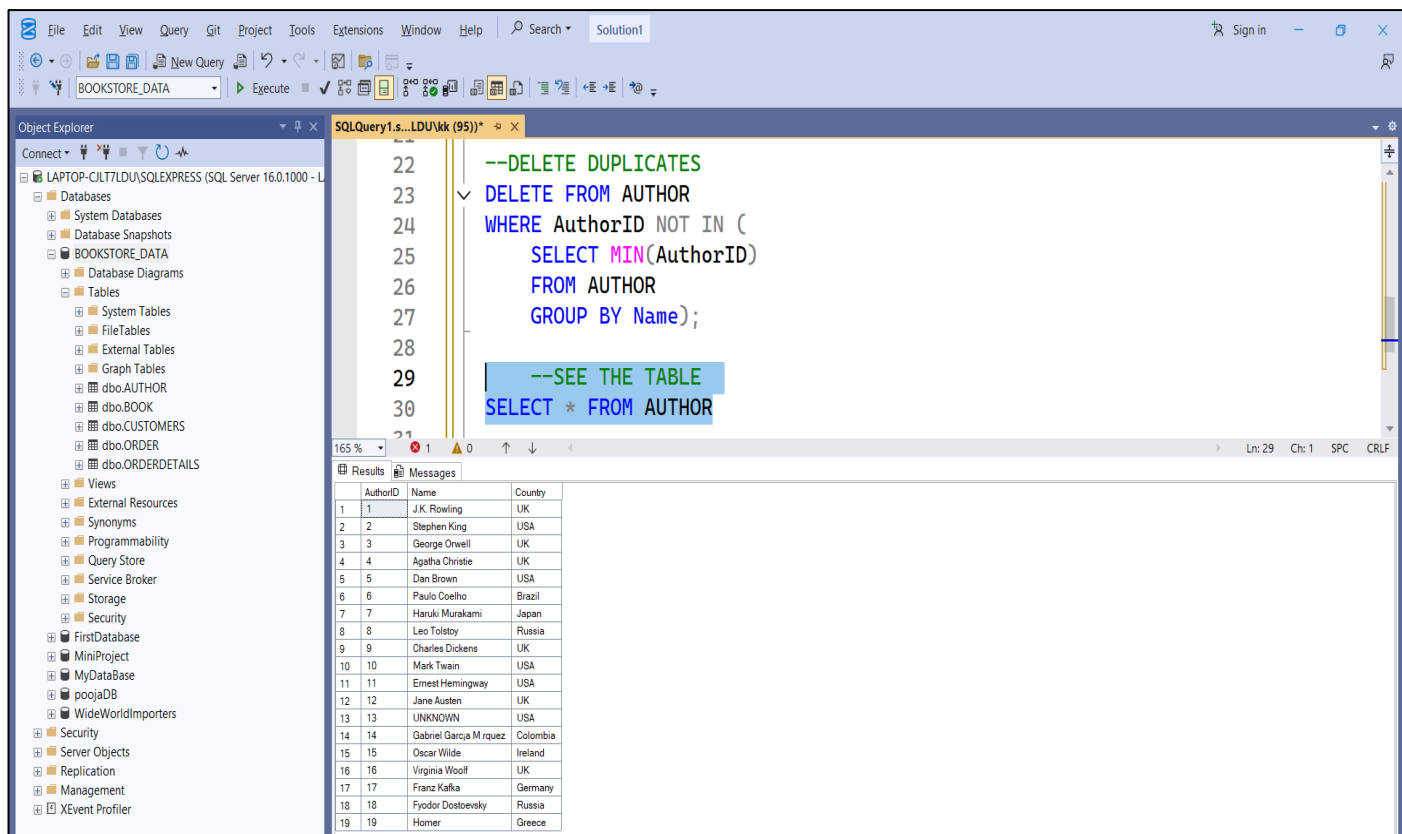


The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'BOOKSTORE\_DATA'. The main window shows a SQL query in the 'SQLQuery1.s...LDU\kk (95)' window. The query is designed to identify and delete duplicate author records. The query text is as follows:

```
16 --DUPLICATE
17 SELECT Name, COUNT(*) AS DuplicateCount
18 FROM AUTHOR
19 GROUP BY Name
20 HAVING COUNT(*) > 1;
21
22 --DELETE DUPLICATES
23 DELETE FROM AUTHOR
24 WHERE AuthorID NOT IN (
25     SELECT MIN(AuthorID)
26     FROM AUTHOR
27     GROUP BY Name);
28
```

The Messages pane at the bottom indicates that 1 row was affected and provides the completion time: 2025-09-13T17:11:48.8226433+05:30.

After handling NULL value and Duplicate Record table look like this-



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'BOOKSTORE\_DATA'. The main window shows a SQL query in the 'SQLQuery1.s...LDU\kk (95)' window. The query is designed to delete duplicate author records and then view the remaining data in the AUTHOR table. The query text is as follows:

```
22 --DELETE DUPLICATES
23 DELETE FROM AUTHOR
24 WHERE AuthorID NOT IN (
25     SELECT MIN(AuthorID)
26     FROM AUTHOR
27     GROUP BY Name);
28
29 --SEE THE TABLE
30 SELECT * FROM AUTHOR
31
```

The Results pane at the bottom displays the data from the AUTHOR table. The data is as follows:

| AuthorID | Name                   | Country  |
|----------|------------------------|----------|
| 1        | J.K. Rowling           | UK       |
| 2        | Stephen King           | USA      |
| 3        | George Orwell          | UK       |
| 4        | Agatha Christie        | UK       |
| 5        | Dan Brown              | USA      |
| 6        | Paulo Coelho           | Brazil   |
| 7        | Haruki Murakami        | Japan    |
| 8        | Leo Tolstoy            | Russia   |
| 9        | Charles Dickens        | UK       |
| 10       | Mark Twain             | USA      |
| 11       | Ernest Hemingway       | USA      |
| 12       | Jane Austen            | UK       |
| 13       | UNKNOWN                | USA      |
| 14       | Gabriel Garcia M rquez | Colombia |
| 15       | Oscar Wilde            | Ireland  |
| 16       | Virginia Woolf         | UK       |
| 17       | Franz Kafka            | Germany  |
| 18       | Fyodor Dostoevsky      | Russia   |
| 19       | Homer                  | Greece   |

## Result:

After cleaning, the database became more accurate and reliable. Reports generated from this data are now trustworthy and help the PageTurners Bookstore owner make better business decisions..

### Books Table —

- **Problem Found:** Missing or invalid values in *Title*, *Genre*, and *Price* columns. Duplicate books existed with the same *Title* and *AuthorID*.
- **Action Taken:**
  - ✓ Updated invalid or placeholder *Title* and *Genre* to “UNKNOWN”.
  - ✓ Replaced missing *Price* values with the average book price.
  - ✓ Removed duplicate records, keeping the first occurrence of each book.
- **Result:** Books table now has complete and consistent values, with no duplicate records.

### Customers Table —

- **Problem Found:** Missing or placeholder values in *Name* and *Country*. Duplicate customers with the same *Name* and *Country*.
- **Action Taken:**
  - ✓ Updated invalid names and countries to “UNKNOWN”.
  - ✓ Removed duplicate customer entries using GROUP BY Name, Country.
- **Result:** Customers table now contains clean and unique customer records.

### Orders Table —

- **Problem Found:** Some rows had missing *OrderDate*. Duplicate orders were found for the same *CustomerID*.
- **Action Taken:**
  - ✓ Updated missing *OrderDate* with a default value (11-Feb-2024).
  - ✓ Deleted duplicate orders, keeping only the earliest record for each customer.
- **Result:** Orders table now has no missing dates and contains only unique order entries.

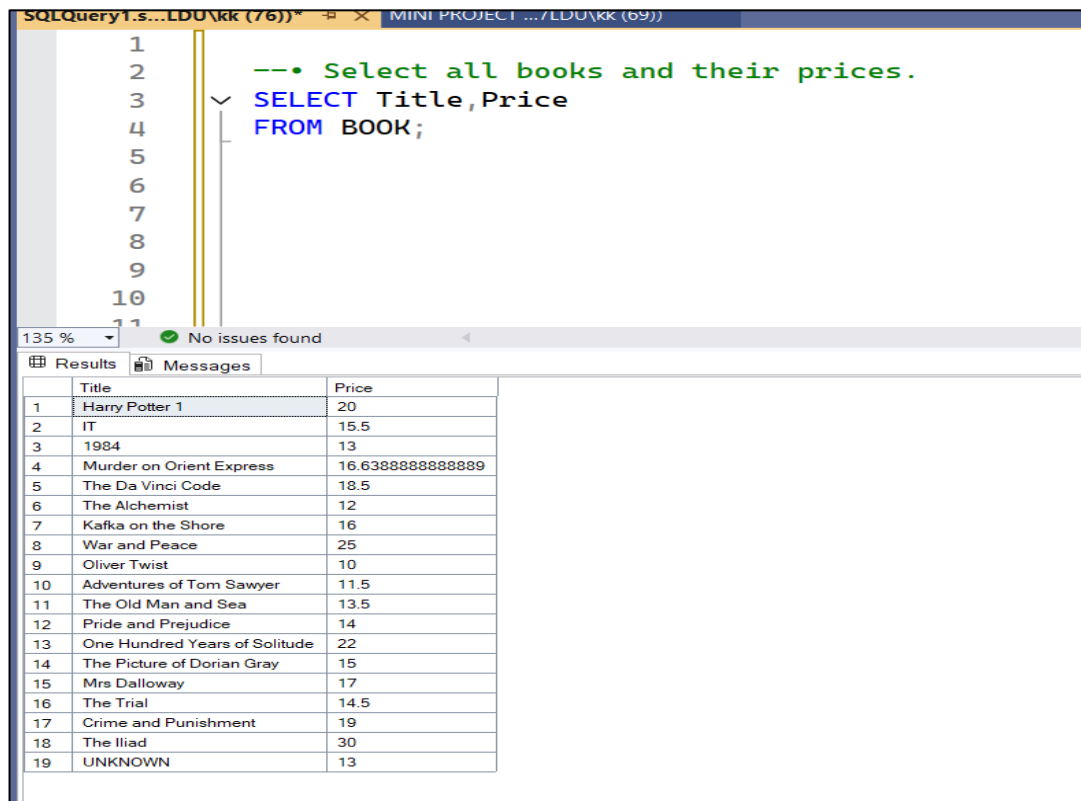
### OrderDetails Table —

- **Problem Found:** Missing values in *Quantity*. Duplicate order details for the same *OrderID*.
- **Action Taken:**
  - ✓ Updated missing quantities with the rounded average of all non-null quantities.
  - ✓ Deleted duplicate order details, keeping one record per order.
- **Result:** OrderDetails table now has valid quantities and no duplicate rows.

## 5. Results and Discussion

After cleaning the data, different SQL queries were executed to answer the PageTurners Bookstore owner's queries. The results are summarized below:

### 1. Books and Prices Query:



The screenshot shows a SQL query window with the following query:

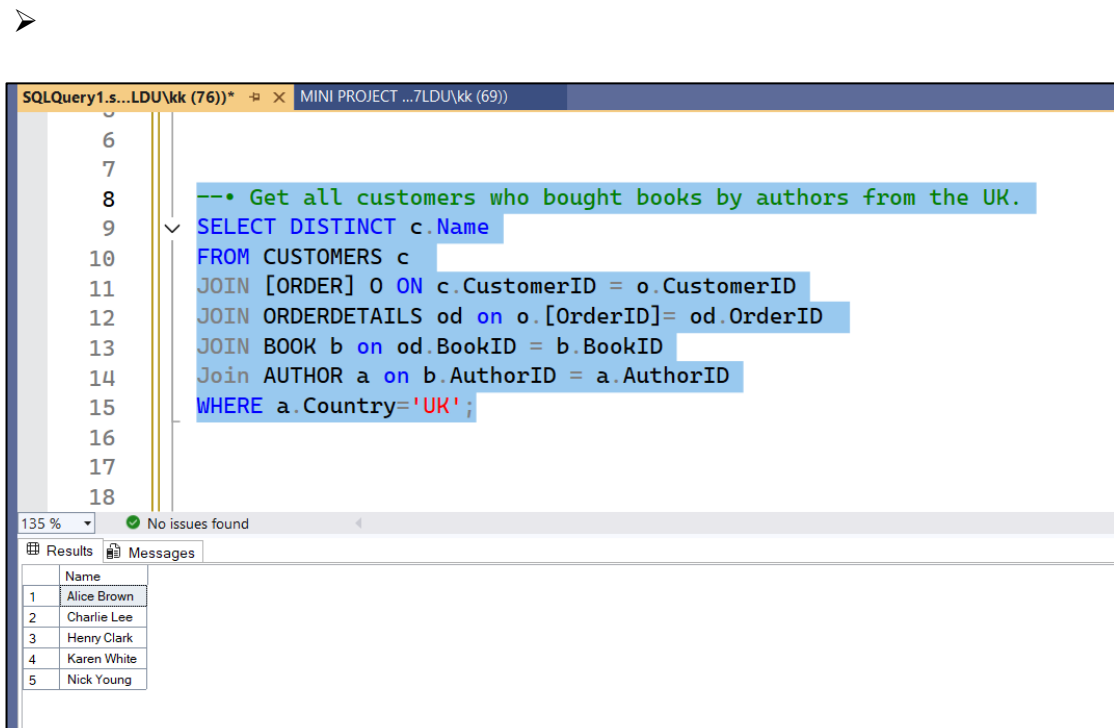
```
--• Select all books and their prices.
SELECT Title, Price
FROM BOOK;
```

The results pane displays a table with 19 rows and 2 columns: Title and Price.

|    | Title                         | Price            |
|----|-------------------------------|------------------|
| 1  | Harry Potter 1                | 20               |
| 2  | IT                            | 15.5             |
| 3  | 1984                          | 13               |
| 4  | Murder on Orient Express      | 16.6388888888889 |
| 5  | The Da Vinci Code             | 18.5             |
| 6  | The Alchemist                 | 12               |
| 7  | Kafka on the Shore            | 16               |
| 8  | War and Peace                 | 25               |
| 9  | Oliver Twist                  | 10               |
| 10 | Adventures of Tom Sawyer      | 11.5             |
| 11 | The Old Man and Sea           | 13.5             |
| 12 | Pride and Prejudice           | 14               |
| 13 | One Hundred Years of Solitude | 22               |
| 14 | The Picture of Dorian Gray    | 15               |
| 15 | Mrs Dalloway                  | 17               |
| 16 | The Trial                     | 14.5             |
| 17 | Crime and Punishment          | 19               |
| 18 | The Iliad                     | 30               |
| 19 | UNKNOWN                       | 13               |

❖ *Insight:* Helps the owner quickly check and compare prices of all titles.

### 2. Customers Who Bought Books by UK Authors



The screenshot shows a SQL query window with the following query:

```
--• Get all customers who bought books by authors from the UK.
SELECT DISTINCT c.Name
FROM CUSTOMERS c
JOIN [ORDER] o ON c.CustomerID = o.CustomerID
JOIN ORDERDETAILS od on o.[OrderID]= od.OrderID
JOIN BOOK b on od.BookID = b.BookID
Join AUTHOR a on b.AuthorID = a.AuthorID
WHERE a.Country='UK';
```

The results pane displays a table with 5 rows and 1 column: Name.

|   | Name        |
|---|-------------|
| 1 | Alice Brown |
| 2 | Charlie Lee |
| 3 | Henry Clark |
| 4 | Karen White |
| 5 | Nick Young  |

❖ *Insight:* Useful for analyzing regional author preferences.



### 3. Total Books per Genre

The screenshot shows a SQL query editor with the following query:

```
--• Show the total number of books per genre
SELECT Genre, COUNT(*) AS TotalBooks
FROM BOOK
GROUP BY Genre;
```

The results pane displays a table with 10 rows and 2 columns: Genre and TotalBooks.

|    | Genre      | TotalBooks |
|----|------------|------------|
| 1  | Epic       | 1          |
| 2  | Fantasy    | 2          |
| 3  | Fiction    | 8          |
| 4  | Historical | 1          |
| 5  | Horror     | 2          |
| 6  | Literary   | 1          |
| 7  | Mystery    | 1          |
| 8  | Romance    | 1          |
| 9  | Thriller   | 1          |
| 10 | UNKWON     | 1          |

❖ *Insight:* Shows stock distribution across genres and highlights which categories need more inventory.

### 4. Books Costing More Than \$14

The screenshot shows a SQL query editor with the following query:

```
--• Find books that cost more than $14.
SELECT Title, Price
FROM BOOK
WHERE Price >14;
```

The results pane displays a table with 12 rows and 2 columns: Title and Price.

|    | Title                         | Price             |
|----|-------------------------------|-------------------|
| 1  | Harry Potter 1                | 20                |
| 2  | IT                            | 15.5              |
| 3  | Murder on Orient Express      | 16.63888888888889 |
| 4  | The Da Vinci Code             | 18.5              |
| 5  | Kafka on the Shore            | 16                |
| 6  | War and Peace                 | 25                |
| 7  | One Hundred Years of Solitude | 22                |
| 8  | The Picture of Dorian Gray    | 15                |
| 9  | Mrs Dalloway                  | 17                |
| 10 | The Trial                     | 14.5              |
| 11 | Crime and Punishment          | 19                |
| 12 | The Iliad                     | 30                |

❖ *Insight:* Helps the owner manage high-value titles separately.

## 5. Customers and the Books They Purchased

```
28
29  --• List customers and the titles of the books they purchased.
30  SELECT c.Name AS Customer, b.Title AS BOOK
31  FROM CUSTOMERS c
32  JOIN [Order] o ON c.CustomerID = o.CustomerID
33  JOIN OrderDetails od ON o.OrderID = od.OrderID
34  JOIN BOOK b ON od.BookID = b.BookID;
35
```

135 % No issues found

Results Messages

|    | Customer      | BOOK                          |
|----|---------------|-------------------------------|
| 1  | Alice Brown   | Harry Potter 1                |
| 2  | Isabel Scott  | Adventures of Tom Sawyer      |
| 3  | Jack Turner   | The Old Man and Sea           |
| 4  | Karen White   | Pride and Prejudice           |
| 5  | Liam Hall     | One Hundred Years of Solitude |
| 6  | Mary Adams    | The Picture of Dorian Gray    |
| 7  | Nick Young    | Mrs Dalloway                  |
| 8  | Olivia Evans  | The Trial                     |
| 9  | Paul Green    | Crime and Punishment          |
| 10 | Queen Taylor  | The Iliad                     |
| 11 | Bob Smith     | IT                            |
| 12 | Charlie Lee   | Murder on Orient Express      |
| 13 | David Johnson | The Da Vinci Code             |
| 14 | Emma Wilson   | The Alchemist                 |
| 15 | Frank Harris  | Kafka on the Shore            |
| 16 | Grace Kim     | War and Peace                 |
| 17 | Henry Clark   | Oliver Twist                  |

❖ *Insight:* Useful for customer management and personalized marketing.

## 6. Books with Authors and Genres

```
33
34  --• List all books along with their author and genre.
35  SELECT b.Title, b.Genre, a.Name
36  FROM BOOK b
37  JOIN AUTHOR a ON b.AuthorID=a.AuthorID;
```

135 % No issues found

Results Messages

|    | Title                         | Genre      | Name                   |
|----|-------------------------------|------------|------------------------|
| 1  | Harry Potter 1                | Fantasy    | J.K. Rowling           |
| 2  | IT                            | Horror     | Stephen King           |
| 3  | 1984                          | Fiction    | George Orwell          |
| 4  | Murder on Orient Express      | Mystery    | Agatha Christie        |
| 5  | The Da Vinci Code             | Thriller   | Dan Brown              |
| 6  | The Alchemist                 | Fiction    | Paulo Coelho           |
| 7  | Kafka on the Shore            | Fantasy    | Haruki Murakami        |
| 8  | War and Peace                 | Historical | Leo Tolstoy            |
| 9  | Oliver Twist                  | Fiction    | Charles Dickens        |
| 10 | Adventures of Tom Sawyer      | Fiction    | Mark Twain             |
| 11 | The Old Man and Sea           | Fiction    | Ernest Hemingway       |
| 12 | Pride and Prejudice           | Romance    | Jane Austen            |
| 13 | One Hundred Years of Solitude | Fiction    | Gabriel Garcia M rquez |
| 14 | The Picture of Dorian Gray    | Fiction    | Oscar Wilde            |
| 15 | Mrs Dalloway                  | Literary   | Virginia Woolf         |
| 16 | The Trial                     | UNKWON     | Franz Kafka            |
| 17 | Crime and Punishment          | Fiction    | Fyodor Dostoevsky      |
| 18 | The Iliad                     | Epic       | Homer                  |
| 19 | UNKNOWN                       | Horror     | Stephen King           |

❖ *Insight:* Provides a complete catalog view for reporting.

## 7. Customers Who Ordered Books Above \$12

```
39  --• Find customers who ordered books costing more than $12.
40  SELECT DISTINCT c.Name
41  FROM CUSTOMERS c
42  JOIN [ORDER] o ON c.CustomerID= o.customerID
43  JOIN [ORDERDETAILS] od ON o.OrderID=od.OrderID
44  JOIN BOOK b ON od.BookID=b.BookID
45  WHERE Price>12;
46
```

135 % No issues found

|    | Name          |
|----|---------------|
| 1  | Alice Brown   |
| 2  | Bob Smith     |
| 3  | Charlie Lee   |
| 4  | David Johnson |
| 5  | Frank Harris  |
| 6  | Grace Kim     |
| 7  | Jack Turner   |
| 8  | Karen White   |
| 9  | Liam Hall     |
| 10 | Mary Adams    |
| 11 | Nick Young    |
| 12 | Olivia Evans  |
| 13 | Paul Green    |
| 14 | Queen Taylor  |

❖ *Insight:* Helps identify premium customers.

## 8. Average Price per Author

```
58
59  --• Find the average price per author.
60  SELECT a.Name AS Author, AVG(b.Price) AS AvgPrice
61  FROM AUTHOR a
62  JOIN BOOK b ON a.AuthorID = b.AuthorID
63  GROUP BY a.Name;
```

134 % No issues found

|    | Author                 | AvgPrice         |
|----|------------------------|------------------|
| 1  | Agatha Christie        | 16.6388888888889 |
| 2  | Charles Dickens        | 10               |
| 3  | Dan Brown              | 18.5             |
| 4  | Ernest Hemingway       | 13.5             |
| 5  | Franz Kafka            | 14.5             |
| 6  | Fyodor Dostoevsky      | 19               |
| 7  | Gabriel Garcia M rquez | 22               |
| 8  | George Orwell          | 13               |
| 9  | Haruki Murakami        | 16               |
| 10 | Homer                  | 30               |
| 11 | J.K. Rowling           | 20               |
| 12 | Jane Austen            | 14               |
| 13 | Leo Tolstoy            | 25               |
| 14 | Mark Twain             | 11.5             |
| 15 | Oscar Wilde            | 15               |
| 16 | Paulo Coelho           | 12               |
| 17 | Stephen King           | 14.25            |
| 18 | Virginia Woolf         | 17               |

❖ *Insight:* Helps in price comparison between authors.

## 9. Top 3 Best-Selling Books

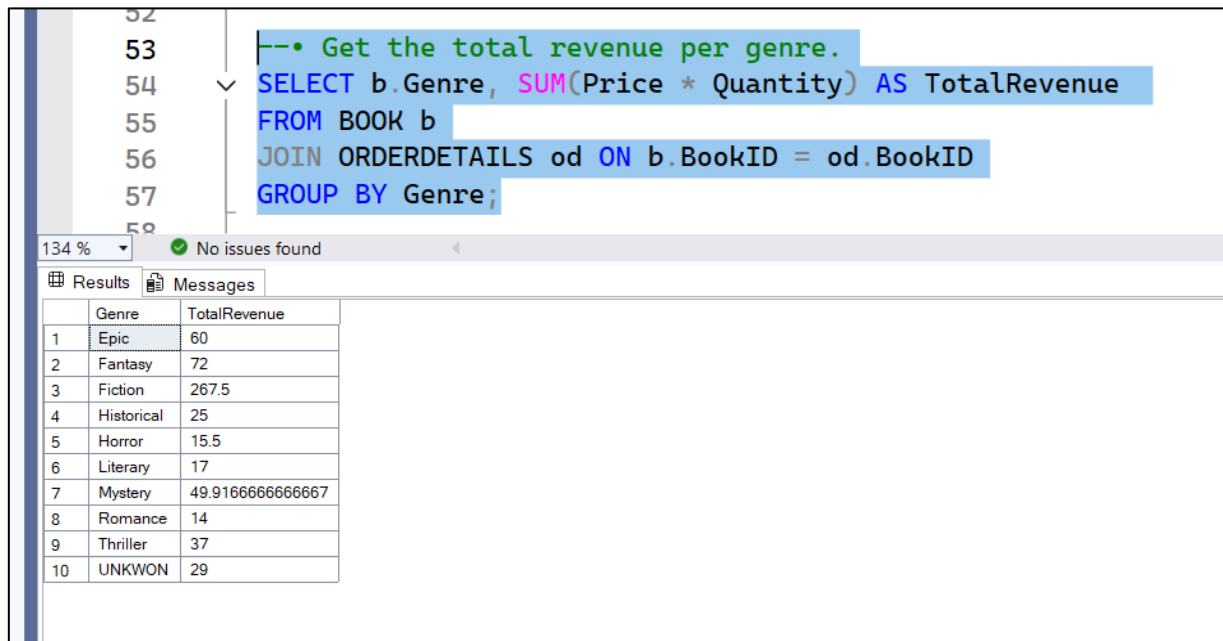
```
46
47  --• Find the top 3 best-selling books by quantity.
48  SELECT TOP 3 b.Title, SUM(od.Quantity) AS TotalSales
49  FROM [ORDERDETAILS] od
50  JOIN BOOK b ON od.BookID=b.BookID
51  JOIN AUTHOR a ON b.AuthorID=a.AuthorID
52  GROUP BY b.Title
53  ORDER BY TotalSales DESC
54
```

135 % No issues found

|   | Title                    | TotalSales |
|---|--------------------------|------------|
| 1 | Oliver Twist             | 5          |
| 2 | The Alchemist            | 4          |
| 3 | Adventures of Tom Sawyer | 3          |

❖ *Insight:* Helps plan restocking and promotions for popular titles.

## 10. Total Revenue per Genre



```
--• Get the total revenue per genre.
SELECT b.Genre, SUM(Price * Quantity) AS TotalRevenue
FROM BOOK b
JOIN ORDERDETAILS od ON b.BookID = od.BookID
GROUP BY Genre;
```

|    | Genre      | TotalRevenue     |
|----|------------|------------------|
| 1  | Epic       | 60               |
| 2  | Fantasy    | 72               |
| 3  | Fiction    | 267.5            |
| 4  | Historical | 25               |
| 5  | Horror     | 15.5             |
| 6  | Literary   | 17               |
| 7  | Mystery    | 49.9166666666667 |
| 8  | Romance    | 14               |
| 9  | Thriller   | 37               |
| 10 | UNKWON     | 29               |

❖ *Insight:* Shows which genres are most profitable for the store.

### ❖ Discussion Summary

The results proved that SQL queries are powerful tools for solving real PageTurners Bookstore problems. The owner can now:

- Track inventory and pricing.
- Identify best-selling books and profitable genres.
- Understand customer buying patterns.
- Manage suppliers effectively.
- Update records with ease.

## 6.Conclusion

This project showed how SQL Server can help manage PageTurners Bookstore data in an organized way. By cleaning the data and fixing errors like missing or duplicate information, the database became accurate and reliable.

Using SQL queries, we were able to answer important business questions, such as which books sold the most, how much revenue each genre made, and who the best customers were. These insights can help store managers make better decisions about stock, sales, and customer services.

Overall, this project proved that SQL is not just for storing data. It is also a powerful tool to understand business trends, solve real problems, and make smarter decisions.

## **7.Appendix**

This appendix contains the complete SQL code, table structures, data cleaning queries, and query r used in the project. It serves as technical documentation to support the main report.

### **A. Database Schema (Table Structures)**

| <b>Author</b> |              |                         |
|---------------|--------------|-------------------------|
| Columns Name  | Data Type    | Other info.             |
| AuthorID      | tinyint      | Primary Key<br>Not NULL |
| Name          | Nvarchar(50) | NULL                    |
| Country       | Nvarchar(50) | Not NULL                |

| <b>BOOK</b>  |              |                         |
|--------------|--------------|-------------------------|
| Columns Name | Data Type    | Other info.             |
| BookID       | tinyint      | Primary Key<br>Not NULL |
| Title        | Nvarchar(50) | NULL                    |
| AuthorID     | tinyint      | Not NULL                |
| Genre        | Nvarchar(50) | NULL                    |
| Price        | Float        | NULL                    |

| <b>Customer</b> |              |                         |
|-----------------|--------------|-------------------------|
| Columns Name    | Data Type    | Other info.             |
| CustomerID      | tinyint      | Primary Key<br>Not NULL |
| Name            | Nvarchar(50) | NULL                    |
| Country         | Nvarchar(50) | NULL                    |





## Book

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'FirstDatabase', including tables 'dbo.ORDER' and 'dbo.ORDERDETAILS'. The main pane shows a SQL script titled 'DataManipulat...LDU\kk (68)' with the following content:

```
32
33 --2. BOOK TABELE MANIPULAT
34 --SEE THE TABLE
35 SELECT * FROM BOOK
36
37 -- Check for empty strings
38 SELECT *
39 FROM BOOK
40 WHERE Title IN ('NULL', 'NA', 'N/A', '-')
41 OR Genre IN ('NULL', 'NA', 'N/A', '-');
42
43 -- Clean Title column
44 UPDATE BOOK
45 SET Title = 'UNKNOWN'
46 WHERE Title IN ('NULL', 'NA', 'N/A', '-');
47
48 -- Clean Genre column
49 UPDATE BOOK
50 SET Genre = 'UNKNOWN'
51 WHERE Genre IN ('NULL', 'NA', 'N/A', '-');
52
53 UPDATE BOOK
54 SET Price = (SELECT AVG(Price) FROM BOOK WHERE Price IS NOT NULL)
55 WHERE Price IS NULL;
56
57 SELECT * FROM BOOK
58
59 -- Find duplicates by Title + AuthorID
60 SELECT Title, AuthorID, COUNT(*) AS DuplicateCount
61 FROM BOOK
62 GROUP BY Title, AuthorID
63 HAVING COUNT(*) > 1;
64
65 --DELETE
66 DELETE FROM BOOK
67 WHERE BookID NOT IN (
68     SELECT MIN(BookID)
69     FROM BOOK
70     GROUP BY Title, AuthorID);
71
72 --SEE THE CLEAN TABLE
73 SELECT *FROM BOOK
74
75
```

## Customer

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'FirstDatabase', including tables 'dbo.ORDER' and 'dbo.ORDERDETAILS'. The main pane shows a SQL script titled 'DataManipulat...LDU\kk (68)' with the following content:

```
76
77 --3. CUSTOMER TABLE MANIPULAT
78 -- SEE THE ROW TABLE
79 SELECT * FROM CUSTOMERS
80
81 -- Check for empty strings
82 SELECT *
83 FROM CUSTOMERS
84 WHERE Name IN ('NULL', 'NA', 'N/A', '-')
85 OR Country IN ('NULL', 'NA', 'N/A', '-');
86
87 -- Clean Name column
88 UPDATE CUSTOMERS
89 SET Name = 'UNKNOWN'
90 WHERE Name IN ('NULL', 'NA', 'N/A', '-');
91
92 -- Clean Country column
93 UPDATE CUSTOMERS
94 SET Country = 'UNKNOWN'
95 WHERE Country IN ('NULL', 'NA', 'N/A', '-');
96
97 --SEE THE TABLE
98 SELECT * FROM CUSTOMERS
99
100 -- Find duplicates by Title + AuthorID
101 SELECT Name, Country, COUNT(*) AS DuplicateCount
102 FROM CUSTOMERS
103 GROUP BY Name, Country
104 HAVING COUNT(*) > 1;
105
106 --DELETE
107 DELETE FROM CUSTOMERS
108 WHERE CustomerID NOT IN (
109     SELECT MIN(CustomerID)
110     FROM CUSTOMERS
111     GROUP BY Name, Country);
112
113 -- SEE THE CLEAN TABLE
114 SELECT *FROM CUSTOMERS
115
```

## Order

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the Object Explorer shows the database structure for 'BOOKSTORE\_DATA'. The 'dbo.ORDER' table is expanded, showing columns: OrderID (PK, nvarchar(50), not null), CustomerID (nvarchar(50), not null), and OrderDate (date, null). The main pane shows a SQL script for 'DataManipulat...LDU\kk (68)' with the following content:

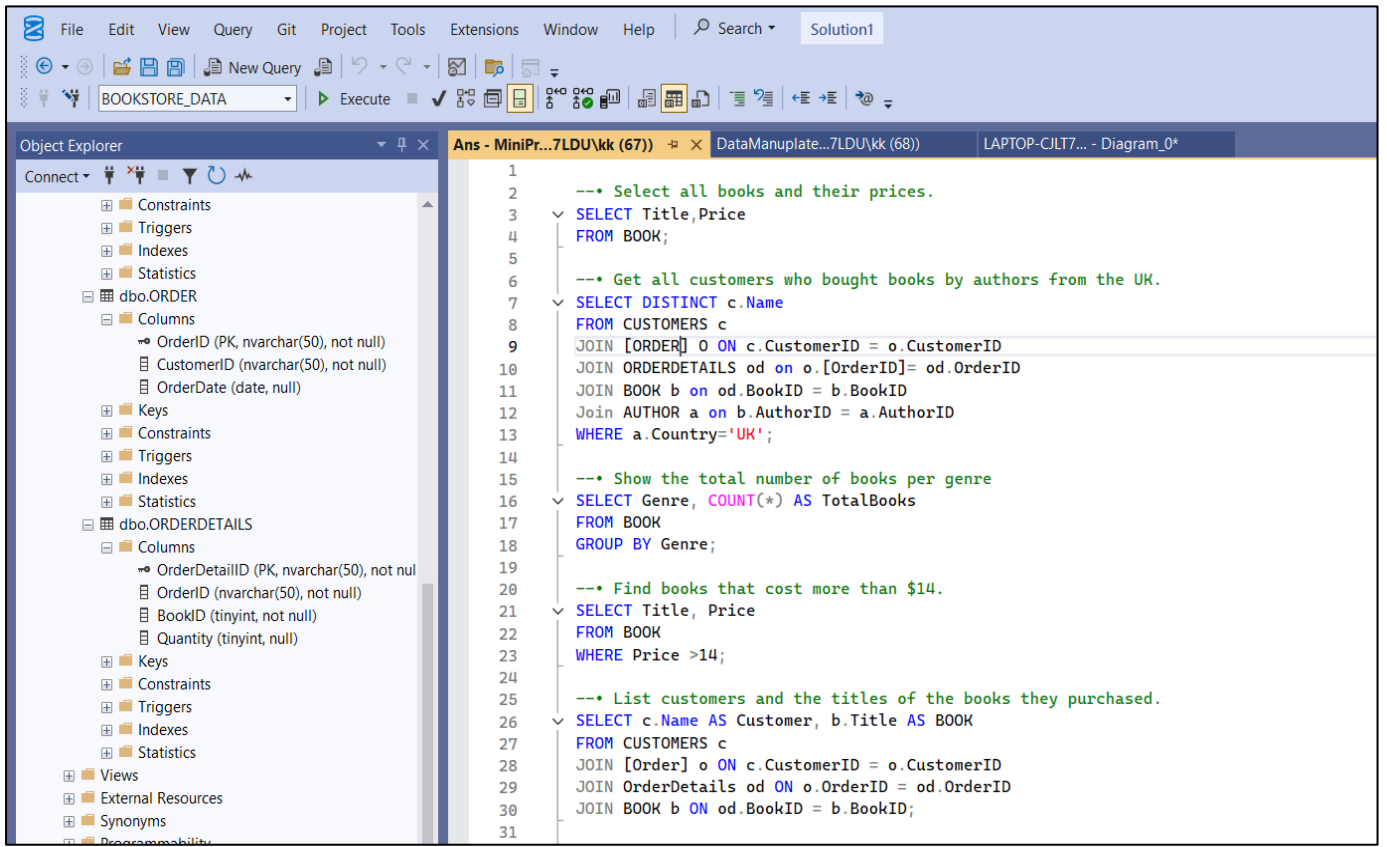
```
115 --4. ORDER TABLE MANIPULAT
116 -- SEE THE ROW TABLE
117 SELECT * FROM [ORDER]
118
119 -- Check NULL
120 SELECT *
121 FROM [ORDER]
122 WHERE OrderDate IS NULL;
123
124 UPDATE [ORDER]
125 SET OrderDate = '2024-02-11'
126 WHERE OrderDate IS NULL;
127
128 --SEE THE TABLE
129 SELECT *FROM [ORDER]
130
131 -- Find duplicates by customerid
132 SELECT CustomerID, COUNT(*) AS DuplicateCount
133 FROM [ORDER]
134 GROUP BY CustomerID
135 HAVING COUNT(*) > 1;
136
137 --DELETE
138 DELETE FROM [ORDER]
139 WHERE OrderID NOT IN (
140 SELECT MIN(OrderID)
141 FROM [ORDER]
142 GROUP BY CustomerID);
143
144 --SEE THE CLEAN TABLE
145 SELECT *FROM [ORDER]
146
```

## OrderDetails

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the Object Explorer shows the database structure for 'BOOKSTORE\_DATA'. The 'dbo.ORDERDETAILS' table is expanded, showing columns: OrderDetailID (PK, nvarchar(50), not null), OrderID (nvarchar(50), not null), BookID (tinyint, not null), and Quantity (tinyint, null). The main pane shows a SQL script for 'DataManipulat...LDU\kk (68)' with the following content:

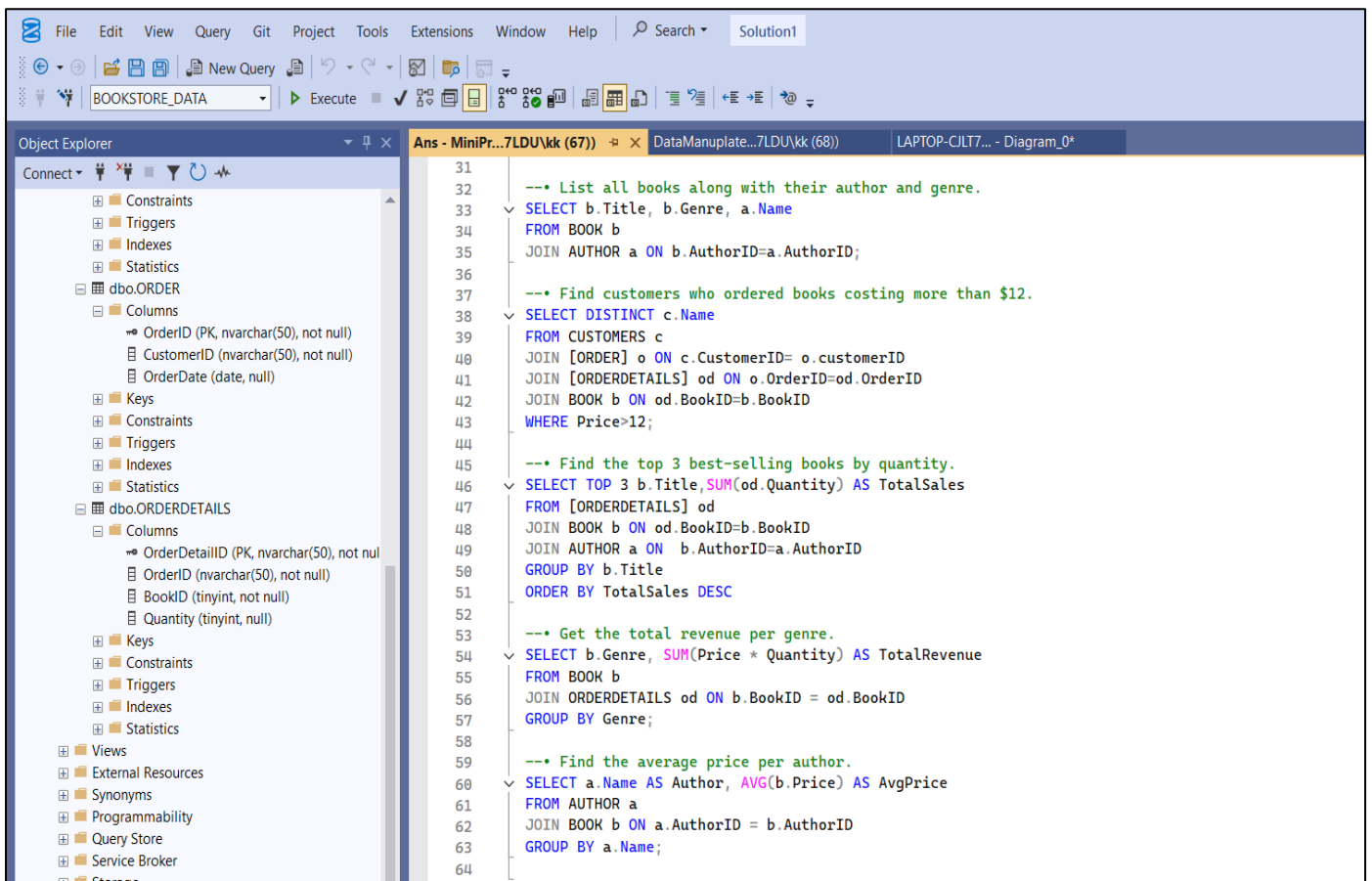
```
147 --5. ORDERDETAIL TABLE MANIPULAT
148 -- SEE THE ROW TABLE
149 SELECT * FROM [ORDERDETAILS]
150
151 -- Check for empty strings
152 SELECT *
153 FROM ORDERDETAILS
154 WHERE Quantity IS NULL;
155
156 UPDATE ORDERDETAILS
157 SET Quantity = ROUND(
158 (SELECT AVG(CAST(Quantity AS FLOAT))
159 FROM ORDERDETAILS
160 WHERE Quantity IS NOT NULL), 0)
161 WHERE Quantity IS NULL;
162
163 --SEE CLEAN TABLE
164 SELECT *FROM ORDERDETAILS
165
166 -- Find duplicates by customerid
167 SELECT OrderID, COUNT(*) AS DuplicateCount
168 FROM ORDERDETAILS
169 GROUP BY OrderID
170 HAVING COUNT(*) > 1;
171
172 --DELETE
173 DELETE FROM [ORDERDETAILS]
174 WHERE OrderDetailID NOT IN (
175 SELECT MIN(OrderDetailID)
176 FROM [ORDERDETAILS]
177 GROUP BY OrderID );
178
179 --SEE THE CLEAN TABLE
180 SELECT *FROM [ORDERDETAILS]
181
```

## C. SQL Queries:



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'BOOKSTORE\_DATA', including tables 'dbo.ORDER', 'dbo.ORDERDETAILS', and their columns and keys. The main query window, titled 'Ans - MiniPr...7LDU\kk (67)', contains the following SQL queries:

```
1
2  --• Select all books and their prices.
3  SELECT Title, Price
4  FROM BOOK;
5
6  --• Get all customers who bought books by authors from the UK.
7  SELECT DISTINCT c.Name
8  FROM CUSTOMERS c
9  JOIN [ORDER] o ON c.CustomerID = o.CustomerID
10 JOIN ORDERDETAILS od ON o.[OrderID] = od.OrderID
11 JOIN BOOK b ON od.BookID = b.BookID
12 JOIN AUTHOR a ON b.AuthorID = a.AuthorID
13 WHERE a.Country='UK';
14
15 --• Show the total number of books per genre
16 SELECT Genre, COUNT(*) AS TotalBooks
17 FROM BOOK
18 GROUP BY Genre;
19
20 --• Find books that cost more than $14.
21 SELECT Title, Price
22 FROM BOOK
23 WHERE Price >14;
24
25 --• List customers and the titles of the books they purchased.
26 SELECT c.Name AS Customer, b.Title AS BOOK
27 FROM CUSTOMERS c
28 JOIN [Order] o ON c.CustomerID = o.CustomerID
29 JOIN OrderDetails od ON o.OrderID = od.OrderID
30 JOIN BOOK b ON od.BookID = b.BookID;
31
```



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'BOOKSTORE\_DATA', including tables 'dbo.ORDER', 'dbo.ORDERDETAILS', and their columns and keys. The main query window, titled 'Ans - MiniPr...7LDU\kk (67)', contains the following SQL queries:

```
31
32 --• List all books along with their author and genre.
33 SELECT b.Title, b.Genre, a.Name
34 FROM BOOK b
35 JOIN AUTHOR a ON b.AuthorID=a.AuthorID;
36
37 --• Find customers who ordered books costing more than $12.
38 SELECT DISTINCT c.Name
39 FROM CUSTOMERS c
40 JOIN [ORDER] o ON c.CustomerID= o.customerID
41 JOIN [ORDERDETAILS] od ON o.OrderID=od.OrderID
42 JOIN BOOK b ON od.BookID=b.BookID
43 WHERE Price>12;
44
45 --• Find the top 3 best-selling books by quantity.
46 SELECT TOP 3 b.Title, SUM(od.Quantity) AS TotalSales
47 FROM [ORDERDETAILS] od
48 JOIN BOOK b ON od.BookID=b.BookID
49 JOIN AUTHOR a ON b.AuthorID=a.AuthorID
50 GROUP BY b.Title
51 ORDER BY TotalSales DESC
52
53 --• Get the total revenue per genre.
54 SELECT b.Genre, SUM(Price * Quantity) AS TotalRevenue
55 FROM BOOK b
56 JOIN ORDERDETAILS od ON b.BookID = od.BookID
57 GROUP BY Genre;
58
59 --• Find the average price per author.
60 SELECT a.Name AS Author, AVG(b.Price) AS AvgPrice
61 FROM AUTHOR a
62 JOIN BOOK b ON a.AuthorID = b.AuthorID
63 GROUP BY a.Name;
64
```

## **8. References**

The following sources were referred to while preparing this project. They provided valuable theoretical concepts, practical examples, and guidance in understanding SQL Server and database management.

1. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th Edition). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th Edition). McGraw Hill.
3. Groff, J. R., & Weinberg, P. N. (2010). *SQL: The Complete Reference*. McGraw Hill.
4. Coronel, C., & Morris, S. (2018). *Database Systems: Design, Implementation, and Management*. Cengage Learning.